Lab Report – 3

CS – 344 | Operating System Lab | 2025
Group No. 24

TASK – 3.1: IMPLEMENT KERNEL PRIMITIVES FOR IPC

Project Objective

The primary objective of this project is to implement Inter-Process Communication (IPC) primitives: **shared memory** and **mailboxes** within the xv6 operating system kernel.

Enhancing xv6 IPC capabilities

The standard xv6 kernel supports only basic IPC through pipes, which enable unidirectional byte streams between related processes but are limited in scope. The two new IPC paradigms to enable more efficient data sharing and flexible process coordination.

1. **Shared Memory:**
    - It introduces a shared memory mechanism where the same physical page is mapped into the virtual address spaces of multiple processes, allowing direct data access without kernel-mediated copying.
2. **Mailboxes:**
    - Mailboxes provide a structured message-passing mechanism that allows processes to send and receive discrete messages asynchronously.

Design and Implementation of Shared Memory

High Level Description

The solution introduces a **Shared Memory Manager** as a new subsystem in the xv6 kernel, designed to support efficient inter-process communication. At its core is a global, fixed-size table (`shm_table`) that tracks all active shared memory regions.

Each entry records the region's key, physical page pointer, reference count, and active status, providing a centralized structure for resource allocation and lookup.

Coarse Locking: To maintain correctness in a multi-processor environment, access to the shared memory table is serialized with a single spinlock, ensuring safe concurrent operations at the cost of some scalability.

Lab Report – 3

CS – 344 | Operating System Lab | 2025
Group No. 24

The manager exposes its functionality to user programs through a set of system calls (`shm_create`, `shm_get`, and `shm_close`), which allow processes to create or attach to shared regions, map them into their address space, and later detach.

Following is the description of these system calls:

- `int shm_create(int key)`: Creates a shared memory region backed by one physical page and associates it with the given `key`. If the key already exists, it returns the existing region's handle instead of creating a new one.

- `void* shm_get(int key)`: Maps the shared page for the given `key` into the calling process's virtual address space and returns a pointer. All processes using the same key share the same physical page.

- `int shm_close(int key)`: Detaches the region from the calling process by unmapping it and reducing the reference count. The physical page is freed only when no processes remain attached.

Implementation

The implementation of the **Shared Memory Manager** is in `kernel/shm.c`

1. Beginning with the definition of the `shm_region` structure. Each `shm_region` represents a single page-sized shared memory segment and stores all necessary metadata:

```
struct shm_region {
  int key;
  void* pa;
  int ref_count;
  int active;
};
```

- **`key`**: An integer provided by the user to uniquely identify the region.
- **`pa`**: A pointer to the physical address of the allocated memory page.
- **`ref_count`**: A reference counter that tracks the number of processes currently attached to this region.
- **`active`**: A flag indicating whether this entry in the shm_table is currently in use.

2. The global `shm_table` is a structure containing this array of regions (regions) and a single spinlock (lock). The choice of a fixed-size array with MAX_SHM_REGIONS set to 64

```
struct {
  struct spinlock lock;
  struct shm_region regions[MAX_SHM_REGIONS];
} shm_table;
```

3. `shminit()` function initializes the Shared Memory Manager by setting up a spinlock for the global `shm_table` and marking all shared memory region entries as inactive.
   This is called in `kernel/main.c`

```
void
shminit(void)
{
  initlock(&shm_table.lock, "shm_table");
  for(int i = 0; i < MAX_SHM_REGIONS; i++) {
    shm_table.regions[i].active = 0;
  }
}
```

4. `shm_create(int key)`: [full code is in attached files]

   - This system call is responsible for creating or retrieving a shared memory region. It begins by acquiring the global `shm_table` lock and checking if a region with the given key already exists. If found, it simply returns the index of that region.
   - If no such region exists, the function looks for a free slot in the table. If the table is full or memory allocation via `kalloc()` fails, it returns -1 to indicate an error.
   - Otherwise, it allocates a zeroed physical page, initializes the region with the provided key, sets its reference count to 0, marks it active, and releases the lock before returning the region's index.
   - This design provides a simple yet reliable way for multiple processes to create and share common memory regions.

5. `shm_get(int key)`: [full code is in attached files]

   - This system call links kernel-managed physical memory with a process's virtual address space. It starts by acquiring `shm_table.lock` to locate the region for the given key. If none exists, it fails.

- Each process maintains a `shm_attach` array to track its attachments, ensuring the same region is not mapped multiple times. If the process is already attached, the existing virtual address is returned; otherwise, the region's `ref_count` is incremented and the global lock is released before page table operations to minimize contention.
- For a new attachment, the kernel finds a free slot in the process's `shm_attach`, chooses the next page-aligned virtual address (`PGROUNDUP(p->sz)`), and maps it to the shared page using `mappages`.
- Error handling is carefully implemented: if mapping fails , the kernel rolls back by re-acquiring the lock and decrementing `ref_count`.

6. `shm_close(int key)`: [full code is in attached files]

- The `shm_close(int key)` system call detaches a shared memory region from a process while managing the lifecycle of the underlying physical page.
- It begins by searching the process's `p->shm_attach` array for the entry with the given key. Once found, it retrieves the virtual address of the mapping, removes the corresponding page table entry with `uvmunmap`, and clears the slot in `p->shm_attach` so it can be reused.
- Next, the function acquires the global `shm_table.lock` to update the shared region's state. It locates the region, decrements its `ref_count`, and checks whether it has reached zero.
- If no processes remain attached, the kernel frees the physical page with `kfree()` and marks the region inactive, making the slot available for reuse.

7. Following changes were made in `proc.h` to help achieving the functionality mentioned in 5 & 6.

```
/*------------ Changes made for Task-3.1 ------------*/
#define SHM_ATTACH_MAX 8

struct shm_attach_entry {
  int key;
  uint64 va;
};
/*---------------------------------------------------*/
```

Added in proc.h

```
/*------------ Changes made for Task-3.1 ------------*/
struct shm_attach_entry shm_attach[SHM_ATTACH_MAX];
/*---------------------------------------------------*/
```

Added in struct proc

These were initiated properly in allocproc() and kfork() in proc.c

8. Desired changes were made in following files to implement syscalls properly: defs.h, sysproc.c, syscall.c, syscall.h, usys.pl, user.h **[codes files are attached at bottom]**

Design and Implementation of Mail-Box

High Level Description

The solution introduces a **Mailbox Manager** as a new subsystem in the xv6 kernel, designed to support reliable message passing for inter-process communication.

At its core is a global, fixed-size table (mbox_table) that maintains all active mailboxes.

Each mailbox entry stores a unique key, a bounded circular buffer of messages, head/tail pointers for indexing, a count of pending messages, and a private spinlock for synchronization.

Coarse + Fine Locking: To ensure correctness in a multiprocessor environment, the global table is protected by a single spinlock for allocation and lookup, while each mailbox has its own lock for concurrent send/receive operations. This reduces contention compared to fully coarse-grained locking.

The mailbox manager exposes its functionality through system calls that allow processes to create mailboxes, send messages, and receive messages:

- `int mbox_create(int key)`: Creates a new mailbox associated with the given key, or returns the handle of an existing one if the key is already in use.
- `int mbox_send(int mbox_id, int msg)`: Appends a message to the mailbox's buffer. If the buffer is full, the caller sleeps until space becomes available.
- `int mbox_recv(int mbox_id, int *msg)`: Retrieves the next message from the mailbox. If the mailbox is empty, the caller sleeps until a message arrives. Messages are safely copied into user space before returning.

Lab Report – 3

CS – 344 | Operating System Lab | 2025
Group No. 24

Implementation

The implementation of the **Shared Memory Manager** is in `kernel/mbox.c`

1. Structure of mailbox: The core of each mailbox is a circular buffer, a highly efficient data structure for implementing queues. The mailbox structure encapsulates this buffer and its associated metadata.

```
struct mailbox {
  int key;
  int active;
  int messages[MBOX_SIZE];
  int head;
  int tail;
  int count;
  struct spinlock lock;
};
```

   - **message**: A fixed-size array that stores the integer messages. With MBOX_SIZE defined as 16, each mailbox has a capacity of 16 messages.
   - **head:** An index pointing to the next message to be received.
   - **tail:** An index pointing to the next available slot for a new message.
   - **count:** The current number of messages in the buffer.
2. The mailbox table: an array of mailboxes with maximum capacity set as MAX_MAILBOXES = 64

```
struct {
  struct spinlock lock;
  struct mailbox mboxes[MAX_MAILBOXES];
} mbox_table;
```

3. `mboxinit()` function initializes the Mail-Box Manager by setting up a spinlock for the global `mbox_table` and marking all shared memory region entries as inactive.
   This is called in `kernel/main.c`

```
void
mboxinit(void)
{
  initlock(&mbox_table.lock, "mbox_table");
  for(int i = 0; i < MAX_MAILBOXES; i++) {
    mbox_table.mboxes[i].active = 0;
    initlock(&mbox_table.mboxes[i].lock, "mailbox");
  }
}
```

4. `mbox_create(int key)`: [full code is in attached files]

   - This system call is responsible for creating a new mailbox or returning an existing one associated with a given key. It begins by acquiring the global `mbox_table.lock` to ensure exclusive access while searching or allocating mailboxes. The function first checks if a mailbox with the same key is already active; if found, it immediately returns the index of that mailbox.

- If no existing mailbox is found, the function searches for an inactive slot in the table. If no free slot exists, it releases the lock and returns -1 to signal failure.
- Otherwise, it initializes the new mailbox by setting the key, marking it active, and resetting the circular buffer pointers (head, tail) and message count to zero. Finally, the lock is released, and the index of the newly created mailbox is returned.

5. `mbox_send(int mbox_id, int msg)`: [full code is in attached files]
   - This system call delivers a message into the mailbox identified by `mbox_id`. It first validates the mailbox ID to ensure it lies within bounds and refers to an active mailbox. Once confirmed, it acquires the mailbox's private lock to safely update its state.
   - If the mailbox buffer is already full (count == MBOX_SIZE), the sending process blocks by calling sleep, waiting until space becomes available.

   - When space exists, the function inserts the new message at the tail of the circular buffer, advances the tail pointer (wrapping around with modulo), and increments the message count.
   - Finally, the lock is released and the function returns success (0). This ensures reliable, blocking message delivery without busy-waiting, while maintaining consistency under concurrent access.

6. `mbox_recv(int mbox_id, int* msg)`: [full code is in attached files]
   - This system call retrieves a message from the mailbox identified by `mbox_id`. It begins with validity checks: ensuring the mailbox ID is within bounds and refers to an active mailbox. Once verified, it acquires the mailbox's lock to safely access its state.
   - If the mailbox is empty (count == 0), the process blocks by calling sleep, waiting until a sender places a message.
   - When a message becomes available, it is read from the buffer at the head index, after which the head pointer is advanced (circularly) and the count is decremented.

- The message is then copied into the user process's memory using `copyout`; if this fails, the lock is released and the call returns an error.

7. Desired changes were made in following files to implement syscalls properly: defs.h, sysproc.c, syscall.c, syscall.h, usys.pl, user.h
   [codes files are attached at bottom]

Test File

Shared Memory [shmtest.c]

This test program demonstrates **shared memory usage between a parent and child process**. It first creates a shared memory region with shm_create(key), then forks. The child attaches to the same region with shm_get(key), waits briefly to let the parent write, and then reads the parent's update (arr[0] = 12). The child writes back a new value (arr[1] = 4) before detaching. Meanwhile, the parent writes into the shared memory, waits for the child to finish, and then verifies the child's update. Finally, both processes call shm_close.

This round-trip of writes and reads confirms that the shared memory is correctly mapped into both processes' address spaces, changes made by one are visible to the other, and proper cleanup occurs after use. It effectively validates the correctness of the shared memory subsystem.

Output:

```
$ shmtest
child saw: 12
parent after child: 4
```

Implementation [mboxtest.c]

This program demonstrates **bidirectional communication between a parent and child process using mailboxes**. It creates two mailboxes (key1 and key2) before forking. The child acts as a receiver on mailbox a: it repeatedly receives messages sent by the parent, then replies back to the parent by sending a response through mailbox b.

Meanwhile, the parent sends a sequence of messages (incrementing s) to the child via mailbox a and then waits for a response on mailbox b. This exchange happens 10 times, after which both processes exit. The test validates that the mailbox subsystem correctly supports blocking send/receive, synchronization, and message passing in both directions between processes.

Lab Report – 3

CS – 344 | Operating System Lab | 2025
Group No. 24

Output: `[it is in attached files]`

TASK – 3.2: THE INTERTWINED MEMORY CHALLENGE

Objective

The objective of this task is to implement an inter-process communication (IPC) mechanism that allows multiple processes to coordinate and share information efficiently. The focus is on enabling processes to exchange data through shared memory and message-passing constructs while ensuring consistency and correctness in communication.

A secondary objective is to design and manage synchronization between processes in such a way that they can follow predefined paths of interaction without conflict. This involves using shared resources to represent common state and dedicated communication channels to exchange signals or messages.

Overall, the task aims to demonstrate the principles of process cooperation in an operating system environment, highlighting the use of shared memory for data exchange and mailboxes for synchronization. This provides a controlled setup to study how processes can work together toward a common goal while operating independently.

Implementation

`master.c:`

- This program acts as the **master process** that sets up the environment for demonstrating inter-process communication (IPC) using shared memory and mailboxes.
- It first creates a shared memory segment and initializes an intertwined path structure where two processes (A and B) will follow alternating positions until reaching an end marker.
- Two mailboxes are then created to enable message passing between these processes—each process uses one mailbox for sending and the other for receiving.
- After setup, the master forks twice to create child processes A and B, passing them the shared memory key, mailbox IDs, starting positions, and roles (send-first or receive-first) via arguments before executing the process program.

- Finally, the master waits for both children to complete, cleans up the shared memory, and reports successful completion.

`process.c:`

- This program represents the worker process logic that participates in the intertwined traversal demonstration using shared memory and mailboxes.
- Each worker (labelled as process A or process B) attaches to a shared memory region that encodes the traversal path.
- The processes coordinate with one another through two mailboxes: one designated for sending messages and the other for receiving.
- Depending on its assigned role—either send-first or receive-first—a process alternates between sending its current position and receiving the other process's position, ensuring proper synchronization.
- Using the received position, it looks up the next step in the shared memory until it encounters the end marker, which terminates the traversal.

Output: `[it is in attached files]`

References

https://pages.cs.wisc.edu/~dusseau/Classes/CS537/Fall2016/Projects/P3/p3.html

https://www.cse.iitb.ac.in/~mythili/os/labs/lab-xv6-mem/xv6-mem.pdf

https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf

Files

https://drive.google.com/drive/folders/1Q18Va9sq647izvvlXKg18_n1ROD3eRLz?usp=sharing